Q-Learning Step-by-step

We take the transition and reward matrix from our previous example (7 LoC)

In[ ]:=
$$T = \left\{ \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ .4 & .4 & 0 & 0 & .2 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix}, \begin{pmatrix} 0 & 0 & .8 & 0 & .2 \\ .4 & .4 & 0 & 0 & .2 \\ 0 & 0 & 0 & .8 & .2 \\ 0 & .5 & 0 & 0 & .5 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix} \right\};$$

In[ ]:=
$$R = \begin{pmatrix} .4 & 0 \\ .2 & 0 \\ .3 & 0 \\ .7 & 0 \\ 0 & 0 \end{pmatrix};$$

We also need to represent Q(s,a) as a matrix with dimension s x a.
Here, we initialise all Q values in an arbitrary way just to make them distinct.
We initalise terminal states to zero as these will never be updates (No action ever takes place in these states)

In[ ]:=
$$Q = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \\ 7 & 8 \\ 0 & 0 \end{pmatrix};$$

Let' s assume we’re in State 1

In[ ]:=
```
S = 1;
```

A single Q-update picks an action in a given state using an epsilon-greedy procedure (so the outcome could be anything). Let’s just do this greedy for now...

We define a policy function that picks the action with the largest q - value for that state

In[ ]:=
```
policy[S_] :=
 FirstPosition[Q⟦S⟧, Max[Q⟦S⟧]] // First
```

In the current state we would then have to choose action 2 (“run”)

In[ ]:=
```
action = policy[state]
```

Out[ ]=

2

We transition according to the probabilities of the transition matrix for action 2 and state 1

In[ ]:=
```
T⟦action, state⟧
```

Out[ ]=

{0, 0, 0.8, 0, 0.2}

Let' s pick a random new state S' according to those transition probabilities

In[ ]:=
```
newState = RandomChoice[T[[action, state]] → Range[5]]
```
Out[ ]=
```
3
```

We now have everything to do the update

In[ ]:=
```
γ = .95; (* interpolation factor *)
```

In[ ]:=
```
α = .1;
```

In[ ]:=
```
Q[[state, action]] + α * (R[[state, action]] + γ Max[Q[[newState]]] - Q[[state, action]])
```
Out[ ]=
```
2.37
```

This is the new value to which we update Q[state,action].

Initialize $Q(s,a), \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$, arbitrarily, and $Q(\textit{terminal-state}, \cdot) = 0$
Repeat (for each episode):
  Initialize S
  Repeat (for each step of episode):
    Choose A from S using policy derived from Q (e.g., ε-greedy)
    Take action A, observe R, S'
    $Q(S,A) \leftarrow Q(S,A) + \alpha\big[R + \gamma \max_a Q(S',a) - Q(S,A)\big]$
    $S \leftarrow S'$;
  until S is terminal

What we have done above is actually SARSA (as we have stayed “on policy”; we have used a single policy for determining the behaviour and for updating) even though SARSA is typically using ϵ-greedy and we have used pure greedy.

If we had used ϵ-greedy for the selection of the next action (behaviour policy) but stayed with the simple “max” operation in the update equation, we’d arrive at true q-learning.

In Python

In[2]:=
```
import numpy as np
```

In[3]:=
```
T=np.array(
    [
        [
            [1, 0, 0, 0, 0],
            [0.4, 0.4, 0, 0, 0.2],
            [0, 0, 1, 0, 0],
            [0, 0, 0, 1, 0],
            [0, 0, 0, 0, 1]
        ],
        [
            [0, 0, 0.8, 0, 0.2],
```

```
            [0.4, 0.4, 0, 0, 0.2],
            [0, 0, 0, 0.8, 0.2],
            [0 , 0.5, 0, 0, 0.5],
            [0, 0, 0, 0, 1]]
    ]
)
```

In[4]:=

```
R=np.array(
    [
        [0.4, 0],
        [0.2, 0],
        [0.3, 0],
        [0.7, 0],
        [0.0, 0]
    ]
)
```

In[5]:=

```
Q=np.array(
    [
        [1, 2],
        [3, 4],
        [5, 6],
        [7, 8],
        [0, 0] # terminal state
    ]
)
```

In[6]:=

```
def policy(S):
    return np.argmax(Q[S])
```

Out[6]= ExternalFunction[System: Python Arguments: {S} IsModule: False IsClass: False]

bear in mind that in Python we have index origin zero, so 0 is actually the first state!

In[10]:=

```
S=0
A=policy(S)
A
```

Out[10]=

1

In[11]:=

```
T[A,S]
```

Out[11]=

NumericArray[Type: Real64 Dimensions: {5} Data: 0., 0., 0.8, 0., 0.2]

In[12]:=

```
S1=np.random.choice(5,p=T[A,S])
S1
```

Out[12]=

2

In[13]:=

```
gamma=0.95
alpha=0.1
Q[S,A]+alpha*(R[S,A]+gamma*np.max(Q[S1])-Q[S,A])
```

Out[13]=

2.37